

Protocole IP

David Darras

24/08/2024

1 L'adressage Internet

- Une **adresse IP** (*Internet Protocol address*) est un identifiant numérique unique attribué à chaque appareil connecté à un réseau tenant sur 32 bits (4 octets). Elle comporte une partie pour identifier le réseau (en rouge-orange) et une partie pour identifier la machine (en jaune). Pour savoir combien de bits sont réservés pour la partie réseau on utilise soit un adressage par classes (A, B, C, D et E), soit un adressage sans classe (CIDR) qui est plus flexible.
- On dit qu'une adresse est publique lorsqu'elle est routable (le cas de votre box par exemple) et privé (souvent débutant par 192.168.X.Y) qu'on peut attribuer à n'importe quel machine sans risque. C'est le NAT (*Network Address Translation*) qui se chargera de vous faire communiquer avec le monde extérieur avec seulement une adresse public.
- **Notation décimale pointée** : 193.27.45.33, soit 11000001 00011011 00101101 00100001 en binaire.
- Un **netmask** se construit en mettant que des 1 dans la partie réseau et des 0 dans la partie machine. Il permet de récupérer la partie réseau d'une adresse IP.
- **Adressage IPv4 par classes** : On utilise les bits de poids fort du premier octet de l'adresse pour connaître la taille de la partie réseau.

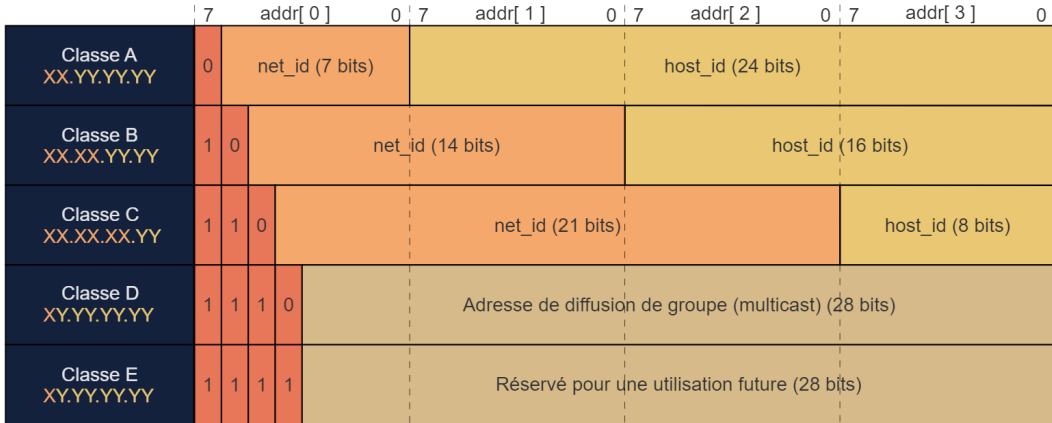


Figure 1: Classes d'adresses Internet

Classe	Netmask	Nombre de réseaux	Nombre d'adresses attribuables	Plage d'adresses
A	255.0.0.0	$2^7 = 128$	$2^{24} - 2 - 1 = 16777214$	0.0.0.0 – 127.255.255.255
B	255.255.0.0	$2^{14} = 16384$	$2^{16} - 2 = 65534$	128.0.0.0 – 191.255.255.255
C	255.255.255.0	$2^{21} = 2097152$	$2^8 - 2 = 254$	192.0.0.0 – 223.255.255.255

Remarque : -2 correspond aux adresses de diffusion et de réseau qui ne peuvent pas être attribuées et -1 correspond à l'adresse de loopback.

```
1 inline bool IsClassA( const u8 addr[ 4 ] ) { return ( addr[ 0 ] & 0x80 ) == 0x00; }
2 inline bool IsClassB( const u8 addr[ 4 ] ) { return ( addr[ 0 ] & 0xC0 ) == 0x80; }
3 inline bool IsClassC( const u8 addr[ 4 ] ) { return ( addr[ 0 ] & 0xE0 ) == 0xC0; }
4 inline bool IsClassD( const u8 addr[ 4 ] ) { return ( addr[ 0 ] & 0xF0 ) == 0xE0; }
5 inline bool IsClassE( const u8 addr[ 4 ] ) { return ( addr[ 0 ] & 0xF8 ) == 0xF0; }
```

- **Addressage IPv4 sans classes :** Le **CIDR** (*Classless Internet Domain Routing*) est une technique d'adressage où on utilise un préfixe pour indiquer le nombre de bits qu'on souhaite pour la partie réseau. Il est généralement stocké sous forme de netmask.
Exemple : 192.168.0.4/9 indique que les 9 premiers bits sont réservés à la partie réseau.

```

1 void CIDR2Netmask( const u8 CIDR, u8 netmask[ 4 ] )
2 {
3     const u32 MASK = ( 0xFFFFFFFF >> CIDR ) ^ 0xFFFFFFFF;
4     for ( u32 i = 0; i < 4; i++ )
5     {
6         netmask[ i ] = ( MASK >> ( 24 - ( 8 * i ) ) ) & 0xFF;
7     }
8 }
9 ///< Exemple: pour CIDR=28, on a netmask[] = { 255, 255, 255, 240 };

```

- **Attention :** Il est interdit de mettre que des 0 dans la partie host pour une station car cela désigne une adresse de réseau. De même, il est interdit de mettre que des 1 car cela désigne l'adresse de broadcast.
- **Adresses spéciales :** Ces adresses ne peuvent pas être donnée à une station.
 - **Adresse de réseau :** Adresse non attribuable pour laquelle la partie **host** est rempli de 0 :
`network_addr = ip_addr & netmask.`
Exemple : L'adresse IP 56.1.73.5/4, soit 0011/1000 000000001 01001001 00000101, appartient au réseau 0011/000 00000000 00000000 00000000, soit 3.0.0.0.
Remarque : On les trouve généralement dans les tables de routage.
 - **Adresse de diffusion (broadcast) :** Adresse non attribuable pour laquelle la partie **host** est rempli de 1 :
`broadcast_addr = ip_addr | ~netmask.`
Exemple : L'adresse IP 176.15.1.16/7, soit 1011000/0 00001111 00000001 00010000 donne l'adresse de diffusion suivante : 1011000/1 11111111 11111111 11111111, soit 177.255.255.255.
 - 127.0.0.1 (*loopback address*) : Elle permet à une station de s'envoyer des données à elle-même sans émettre sur le réseau.
 - 0.0.0.0/0 est utilisée par une machine pour connaître son adresse IP pendant une procédure d'initialisation utilisé par DHCP. De plus, elle aussi utilisé comme **route par défaut** dans les tables de routage.
 - 255.255.255.255 (*limited broadcast*) permet d'envoyer un paquet sur un réseau en mode diffusion si on ne connaît pas l'adresse de réseau sur lequel on se situe.

	7	addr[0]							7	addr[1]							7	addr[2]							7	addr[3]							0
Adresse IP 192.168.0.13/11	1	1	0	0	0	0	0	0	1	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	1		
Netmask 255.224.0.0	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
Adresse réseau 192.160.0.0	1	1	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
Broadcast 192.191.255.255	1	1	0	0	0	0	0	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1		

Figure 2: Obtention de l'adresse de réseau et de l'adresse de diffusion avec le netmask

- **Adressage par sous-réseau :** L'**ICAN** (*Internet Corporation for Assigned Names and Numbers*) vous attribue l'adresse de réseau 128.32.0.0 avec son netmask 255.255.0.0. Or, on souhaite créer plusieurs sous-réseaux distincts. L'idée est de prendre des bits de la partie hôte pour créer une partie sous-réseau (**subnet_id**). Pour séparer la partie réseau et sous-réseau de la partie hôte, on utilise un **VLSM** (*Variable-Length Subnet Mask*).
Exemple :

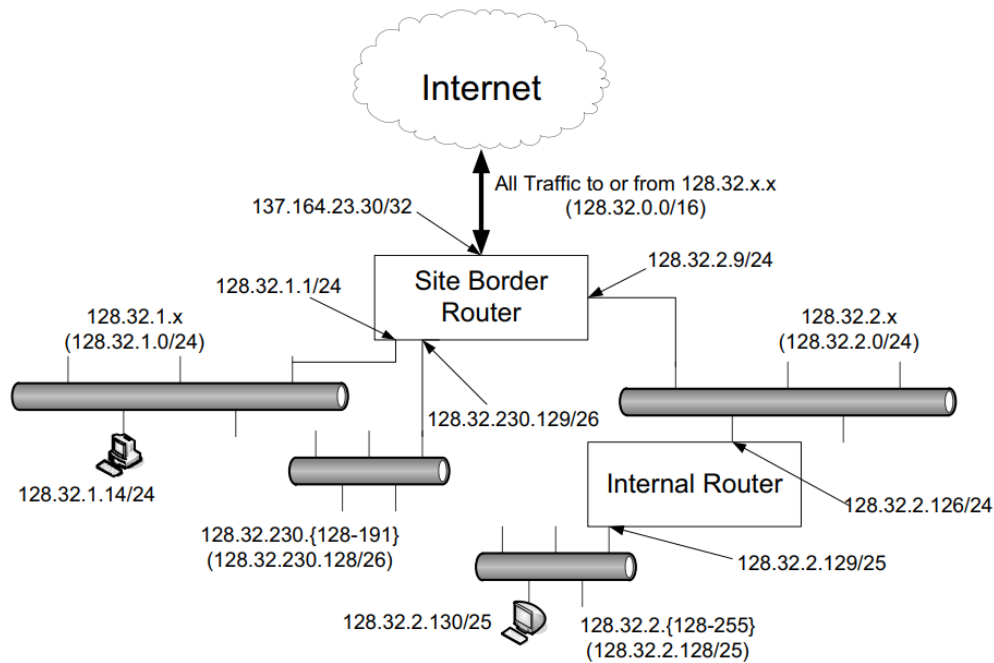


Figure 3: Adressage par sous-réseau

- Les **règles d'agréations** permet de réduire le nombre d'informations contenues dans les tables de routage en regroupant plusieurs annonces de préfixes.

Exemple : 193.44.76.0/24, 193.44.77.0/24, 193.44.78.0/24 et 193.44.79.0/24 peut être regrouper en une seule adresse : 193.44.76.0/22

2 Protocole IP

```

1 struct IPHeader
2 {
3     u8 version : 4;          ///< Version : 4 (IPv4) ou 6 (IPv6)
4     u8 header_length : 4;    ///< Taille de l'en-tête (IHL) : 5 (pour 4x5 = 20 octets) pour un champ option vide.
5                               ///< Attention: L'en-tête est alignée sur 32 bits (4 octets).
6     u8 type_of_service;      ///< Type de service (ToS) : 0 (donne une priorité au paquet mais n'est pas utilisé)
7     u16 total_length;        ///< Taille totale du paquet : en-tête + données
8                               ///< Permet de supprimer les bits de bourrage
9     u16 identification;      ///< Si le paquet doit être fragmenté tous les fragments porteront ce numéros
10    u16 _ : 1;               ///< Inutilisé : 0
11    u16 dont_fragment : 1;    ///< 0 => Fragmentation autorisée par un routeur intermédiaire sinon un paquet ICMP sera envoayit.
12    u16 more_fragments : 1;   ///< 0 => C'est le dernier fragment du datagramme
13    u16 fragment_offset : 13; ///< Position du premier octet dans le datagramme total non fragmenté (multiple de 8 octets sauf pour le dernier)
14    u8 time_to_live;          ///< TTL = 64 par défaut - Nombre maximal de routeurs
15                               ///< que le datagramme peut traverser.
16                               ///< Il est décrémenté chaque fois qu'on traverse un routeur.
17                               ///< Si TTL = 0, on détruit le paquet et envoie un paquet ICMP à l'expéditeur
18                               ///< On évite de faire tourner des paquets en rond et de consommer de la bande passante
19    u8 protocol;              ///< ICMP (1), IGMP (2), TCP (6), UDP (17), ESP (50), OSFP (89)
20    u16 header_checksum;      ///< Vérifie l'intégrité des informations de l'en-tête. (supprimé de IPv6)
21                               ///< Le paquet est détruit si le CRC est incorrect.
22                               ///< Il est recalculé à chaque routeur vu qu'on modifie le champ TTL.
23    u8 source[4];             ///< Adresse IP de l'expéditeur
24    u8 destination[4];        ///< Adresse IP du destinataire
25    ///< Champs option (souvent vide)
26 };

```

- **Définition :** Protocole de la couche réseaux (niveau 3) qui permet d'acheminer des datagrammes d'un réseau à l'autre à l'aide de routeurs et de fragmenter les paquets trop gros pour un réseau. Typiquement, des données de plus de 1500 octets seront fragmentés sur Ethernet comme c'est la limite imposée. Il assure un service **non fiable** car il n'existe **aucune garantie que les datagrammes arrivent à destination**. De plus, certains peuvent être perdus, dupliqués, retardés, altérés ou remis dans le désordre. Le mode de transmission est **non connecté** car **IP traite chaque datagramme indépendamment** de ceux qui le précèdent et le suivent. Dès lors, les datagrammes peuvent ne pas suivre le même chemin pour arriver à destination.

- **Remarque** : Un paquet avec option pénaliserait le réseau car c'est une couche logiciel qui doit s'en occuper avec des copies mémoires, il n'est donc jamais utiliser.
- La MTU (*Maximum Transport Unit*) peut varier d'un réseau à l'autre, sur Ethernet elle est de 1518 octets.
- Si le routeur reçoit un paquet trop gros, il doit s'adapter aux support auquel il va le transmettre en le fragmenteant en plusieurs datagrammes. C'est la station réceptrice de réassembler les datagrammes pour former le datagramme initial.
- Si un paquet se perd on peut tout jeter.
- Les datagrammes fragmentés peuvent suivrent des routes différentes donc un routeur ne peut pas les réassembler entre deux, seul le destinataire peut le faire.
- La **découverte MTU** permet d'avoir la taille minimale du paquet sans avoir à fragmenté plusieurs fois un meme paquet. On met le bit don't framagnet (DF) à 1 et si on reçoit un paquet ICMP avec la taille attendu. Or la route peut changer il faut donc tester périodiquement la MTU n'a pas varié.
- Un **tunnel** transporte un protocole de niveau 3 dans un autre portcole de niveau 3. Exempel : Construire des réseaux privés virtuels (RPV) ou **VPN** (*Virtual Private Network*). On peut ajouter une option pour chiffrer les données. Transporter des paquets en utilisant un adressage privé dans un réseau public via un routeur qui a une adresse publique. Permet aussi de transporter le trafic IPv6 sur une infrastructure IPv4 (appelé 6bone). Encapsulation GRE (*Generic Routing Encapsulation*)
- Le programme **ping** évalue la latence entre un hôte client et un hôte cible. Pour ce faire, il mesure le RTT et enregistre la perte de paquets, puis calcule un résumé statistique de plusieurs échanges ping-pong (le minimum, la moyenne, le maximum et l'écart-type des temps aller-retour). Dès lors, Ping test à distance si un hôte particulier est opérationnel et accessible. Parfois, l'heure locale incluse par le programme **ping** lors d'un Echo Request est renvoyée dans un message Echo Reply. Dès lors, il note l'heure actuelle à laquelle une réponse est reçue et soustrait les deux pour obtenir une estimation du RTT. Étant donné que seule la notion d'heure actuelle de l'expéditeur d'origine est utilisée, cette fonctionnalité ne nécessite aucune synchronisation entre les horloges de l'expéditeur et du destinataire. Si l'on reçoit un paquet ICMP de type 3 (*Destination Unreachable*), on peut en déduire que la station pingée n'est pas connectée au réseau.